

Understanding L2 Regularization in Neural Networks

L2 regularization (also known as Ridge regularization or weight decay) is a technique used to prevent machine learning models from overfitting by adding a penalty to the loss function based on the size of the model's weights.

The Math in Practice

In your code, the regularization term is added as:

$$\text{loss} = \dots + 0.01 \times (W^2).\text{mean}()$$

- (W^2) : Squares every weight in matrix W . This ensures that positive and negative large weights are treated equally, heavily penalizing large magnitudes.
- $\text{.mean}()$: Averages the squared weights to maintain scale.
- 0.01 : The regularization strength (λ). It balances fitting the data versus keeping weights small.

The Gradient and Optimization

To understand how this affects learning, consider the gradient of the total loss L with respect to the weights W :

$$\partial L / \partial W = \partial L_{\text{data}} / \partial W + 2\lambda W$$

When updating weights W using gradient descent with learning rate η :

$$W_{\text{new}} = W_{\text{old}}(1 - 2\eta\lambda) - \eta(\partial L_{\text{data}} / \partial W)$$

Key Effects

- **Weight Shrinkage:** The term $(1 - 2\eta\lambda)$ acts as a multiplier slightly less than 1. This forces weights to continuously decay toward zero, even if the data loss gradient is minimal.
- **Smoothness:** By penalizing extremes, the optimizer is forced to distribute the "learning" more evenly across weights rather than relying on a few massive, noise-fitting values.
- **Better Generalization:** The result is a more robust model that generalizes better to unseen data.